Cline — Configuration Report

Prepared for **Murpheux** · 28 September 2026 · Verified against **Cline extension 4.1.21** (legacy + next builds) and **Cline CLI 3.0.65**
Folder: `~/agent-configs/cline/` → `Documents/Cline/` (both builds) and `~/.cline/` (next + CLI)

1. Summary

Since 4.1.0 one extension package ships **two builds** with different config homes: `legacy` and SDK-based `next`, which shares everything with the CLI. The configuration targets both:

- **Both builds:** rules, workflows and hooks, in `Documents/Cline`, which both builds read.
- **next + CLI:** skills, defined subagents, the guard and MCP, in `~/.cline`.

Routing uses Plan mode (strong model) and Act mode (workhorse), plus defined subagents. Cline subagents **don't inherit approval settings**, so every one is read-only by design; all mutations happen in the Act session, where approvals and the hook apply.

4	16	20	2	5
read-only subagents	workflows	skills	hook launchers	MCP servers

2. Sources used

- `cline/cline`: `main` (SDK, CLI, next) and the `legacy-extension` branch source, docs/ (rules, workflows, skills, hooks, MCP, plan-and-act, auto-approve, subagents, deprecations, CLI reference), and CHANGELOG. Also docs.cline.bot.
- **JhonMA82/awesome-clinerules** (reviewed; see §6) and `cline/prompts`.

3. What was built

File	Purpose
`Documents/Cline/Rules/00-murpheux.md`	full global instructions, with the router policy inlined because the main task is the router. This is the only always-on context
`Documents/Cline/Workflows/*.md`	16 workflows (`/review.md`, `/plan.md`, `/security-scan.md`, …) with per-role instructions (Plan vs Act vs subagent)
`Documents/Cline/Hooks/PreToolUse` · `windows/PreToolUse.ps1`	guard launchers for macOS/Linux/WSL and Windows; both accept the legacy (`preToolUse`) and next (`tool_call`) payloads
`~/.cline/agents/*.yaml`	explorer and reviewer (LiteLLM `deepseek-flash`), architect (OpenRouter `openai/gpt-6-sol`), security (OpenRouter `anthropic/claude-opus-5.5`). Read-only tools only
`~/.cline/skills/`, `~/.cline/guard/`	20 skills (about 100 tokens each until used) and the shared guard
`~/.cline/data/settings/cline_mcp_settings.json`	github (Docker, read-only, 5 toolsets) and context7, both via `op run`; mslearn (streamableHttp); awsdocs; terraform
`~/.cline/cline.op.env`, `mcp.op.env`	1Password references (`LITELLM_API_KEY`, `OPENROUTER_API_KEY`, MCP tokens) plus the legacy `CLINE_COMMAND_PERMISSIONS` deny list
`templates/`	`.clineignore` (legacy build) and a VS Code setting to pin the `next` build

4. Routing & tiers

Role	Where	Model
router · planner	Plan mode (only you switch Plan → Act)	OpenRouter `openai/gpt-6-sol` · high
developer · sre	Act mode, approvals on	LiteLLM `claude-sonnet-5` · medium
explorer · reviewer	`subagent_explorer` / `subagent_reviewer`	LiteLLM `deepseek-flash`
architect	`subagent_architect`	OpenRouter `openai/gpt-6-sol`
security	`subagent_security`	OpenRouter `anthropic/claude-opus-5.5`

Provider and model choices live in Cline's UI, not in files. The README lists the exact values. Keys stay out of `providers.json`, which is plaintext: launch VS Code or the CLI with `op run --env-file ~/.cline/cline.op.env -- code .`.

5. Guardrails

1. **Guard hook** on both builds cancels destructive, secret-reading, force-push and interactive commands, and secret-file access through `read_files`, `editor` and `apply_patch`. It was tested with both payload shapes and with Windows PowerShell.
2. **Read-only subagents**, because they bypass approvals.
3. **Approvals**: Edit and Execute set to require approval. `next` defaults to auto-approving edits and MCP, so this has to be changed in the UI.
4. **Legacy extras**: the `CLINE_COMMAND_PERMISSIONS` deny list (deny wins, with no allow list so everything else still prompts) and `.clineignore`, which is not a security boundary.
5. **Credentials**: MCP servers are launched through `op run`; providers read env keys, so nothing secret is stored.

6. awesome-clinerules assessment

The list is a copy of awesome-cursorrules: 131 `.cursorrules` folders, last updated January 2025, and many files are squashed onto a single line. It has nothing for .NET, PowerShell, bash, Terraform/Ansible, cloud, security or diagrams, and its `globs:` frontmatter is ignored by Cline (which uses `paths:`).

Ideas adopted into the global rule and the skills:

- verify before assuming, and make the minimal change;
- answer-length control (fewer output tokens);
- respect the existing architecture;
- stdlib-first Go;
- uv, ruff and pytest for Python;
- strict TypeScript;
- the project-notes / memory-bank pattern.

The gaps were filled from `cline/prompts` (security audit, Mermaid plans, memory bank) and the shared skills.

7. Gotchas handled

- **Two builds, two config worlds**: files are placed where both builds read them, and pinning `next` is recommended.
- **Linux Documents directory**: it is resolved via `xdg-user-dir`. On Windows `install.ps1` uses the real `MyDocuments` folder, which handles OneDrive redirection.
- **Windows hooks**: only `PreToolUse.ps1` runs, and the Windows installer removes the POSIX launcher. Unsigned scripts on UNC paths are blocked, so the installer is run from a local copy.

- **Defaults to watch**: the CLI defaults to `--auto-approve true`, and the `next` build dropped the per-tool MCP `autoApprove` list.

8. Validation

All frontmatter and JSON parse; the 4 subagent definitions have the required keys. The installer smoke test showed the hook cancelling a force-push (next payload) and allowing `go test` (legacy payload), with MCP paths rendered. **On Windows**, the real `PreToolUse.ps1` blocked `Remove-Item -Recurse -Force C:\` and allowed `dotnet test`.

9. Install

```
cd ~/agent-configs/cline && DRY_RUN=1 ./install.sh && ./install.sh     # macOS / WSL
pwsh -File install.ps1                                                 # Windows (from a local copy)
op run --env-file ~/.cline/cline.op.env -- code .                      # launch with injected keys
```